



FACULDADE DE TECNOLOGIA SENAI GASPAR RICARDO JÚNIOR

**DESENVOLVIMENTO DE UM SISTEMA SUPERVISÓRIO WEB PARA
MONITORAMENTO INDUSTRIAL UTILIZANDO OPC-UA,
NODE-RED E NEXT.JS**

**DEVELOPMENT OF A WEB-BASED SUPERVISORY SYSTEM FOR
INDUSTRIAL MONITORING USING OPC-UA, NODE-RED AND
NEXT.JS**

Victor Geroto ¹

²

RESUMO

Este trabalho apresenta o desenvolvimento de um sistema supervisório Web para monitoramento e controle de processos industriais, integrando tecnologias modernas de desenvolvimento Web com protocolos de comunicação industrial. O sistema implementa uma arquitetura em quatro camadas: um servidor OPC-UA simulado em Python para geração de dados de processo (temperatura, pressão e status de operação), o middleware Node-RED para aquisição e roteamento de dados via protocolo OPC-UA, um backend em Express.js (Node.js) responsável pelo armazenamento de históricos e gerenciamento de alarmes, e uma interface Web desenvolvida em Next.js com dashboards de tempo real, gráficos de tendência e telas de diagnóstico de alarmes. Complementarmente, um backend em .NET com banco de dados PostgreSQL gerencia o cadastro de usuários, dispositivos e sensores via API RESTful. A metodologia envolveu a criação de um pipeline completo de aquisição de dados industriais (OPC-UA → Node-RED → Express → Next.js), seguindo boas práticas de alarmística industrial e princípios de interfaces de alta performance conforme a norma ISA-101. Os resultados demonstram a viabilidade da solução para aplicações de supervisão industrial em

ambientes educacionais e de prototipagem, com capacidade de monitoramento em tempo real, registro de históricos de processo e gestão de alarmes.

Palavras-chave: Sistema Supervisório; OPC-UA; Node-RED; IoT Industrial; Next.js; Alarmes Industriais.

ABSTRACT

This paper presents the development of a Web-based supervisory system for monitoring and controlling industrial processes, integrating modern Web development technologies with industrial communication protocols. The system implements a four-layer architecture: a simulated OPC-UA server in Python for generating process data (temperature, pressure, and operating status), the Node-RED middleware for data acquisition and routing via OPC-UA protocol, an Express.js (Node.js) backend responsible for historical data storage and alarm management, and a Web interface built with Next.js featuring real-time dashboards, trend charts, and alarm diagnosis screens. Additionally, a .NET backend with a PostgreSQL database manages the registration of users, devices, and sensors through a RESTful API. The methodology involved creating a complete industrial data acquisition pipeline (OPC-UA → Node-RED → Express → Next.js), following industrial alarm management best practices and high-performance interface principles according to the ISA-101 standard. The results demonstrate the feasibility of the solution for industrial supervision applications in educational and prototyping environments, with capabilities for real-time monitoring, process history recording, and alarm management.

Keywords: Supervisory System; OPC-UA; Node-RED; Industrial IoT; Next.js; Industrial Alarms.

1 Introdução

A transformação digital no setor industrial tem impulsionado a adoção de tecnologias baseadas na Internet das Coisas Industrial (IIoT) e sistemas supervisórios Web, substituindo gradativamente as soluções SCADA tradicionais por plataformas mais flexíveis, acessíveis e interoperáveis (AGUIAR, 2023). Nesse contexto, a integração entre protocolos de comunicação industrial, como o OPC-UA, e frameworks modernos

de desenvolvimento Web representa uma abordagem promissora para a construção de interfaces homem-máquina (IHM) de alta performance.

O protocolo OPC-UA (*Open Platform Communications – Unified Architecture*) destaca-se como padrão para comunicação entre equipamentos industriais, oferecendo interoperabilidade, segurança e modelagem de dados orientada a objetos (OPC FOUNDATION, 2017). Quando combinado com middleware como o Node-RED, torna-se possível criar pipelines de aquisição de dados que conectam o chão de fábrica a aplicações Web em tempo real.

Este trabalho tem como objetivo o desenvolvimento de um sistema supervisório Web completo, capaz de:

- Adquirir dados de um servidor OPC-UA simulado via middleware Node-RED;
- Armazenar e processar dados de processo em um backend Express.js;
- Implementar mecanismos de alarmística industrial com reconhecimento e histórico;
- Apresentar informações em uma interface Web com dashboards de tempo real, gráficos de tendência e telas de diagnóstico, seguindo princípios da norma ISA-101;
- Gerenciar cadastros de usuários, dispositivos e sensores por meio de uma API CRUD em .NET com banco PostgreSQL.

O sistema foi desenvolvido como projeto prático da disciplina de Interfaces Industriais, contemplando os módulos de comunicação industrial, backend supervisório, alarmística e design de IHM.

2 Revisão de Literatura

2.1 Sistemas Supervisórios e SCADA

Sistemas supervisórios são ferramentas essenciais para o monitoramento e controle de processos industriais. Tradicionalmente implementados como sistemas SCADA (*Supervisory Control and Data Acquisition*), essas soluções vêm evoluindo para arquiteturas baseadas na Web, permitindo acesso remoto e maior flexibilidade na construção de interfaces (AGUIAR, 2023).

Segundo Galloway e Hancke (2013), os sistemas SCADA modernos tendem a incorporar padrões abertos de comunicação e interfaces baseadas em navegador, reduzindo a dependência de softwares proprietários e facilitando a integração com ecossistemas de IoT.

2.2 OPC-UA como Padrão de Comunicação Industrial

O OPC-UA é um protocolo de comunicação independente de plataforma, projetado para ambientes industriais. Diferentemente de protocolos anteriores como o OPC-DA (baseado em COM/DCOM e restrito ao Windows), o OPC-UA oferece comunicação segura, modelagem de dados orientada a objetos e suporte a múltiplos padrões de transporte [?].

“A arquitetura OPC-UA foi projetada para fornecer uma infraestrutura robusta e segura para a troca de dados em aplicações industriais, eliminando as limitações de plataforma e promovendo a interoperabilidade entre dispositivos de diferentes fabricantes” (OPC FOUNDATION, 2017).

A estrutura do OPC-UA baseia-se em nós (*nodes*), que representam variáveis, objetos e métodos. Cada nó possui um identificador único (*NodeId*) que permite leitura e escrita de valores de forma padronizada.

2.3 Node-RED como Middleware Industrial

O Node-RED é uma ferramenta de programação visual baseada em fluxos, desenvolvida originalmente pela IBM para aplicações de IoT. Sua arquitetura permite a criação de pipelines de aquisição de dados conectando nós de entrada (sensores, protocolos), processamento (transformação, filtragem) e saída (APIs, bancos de dados) (NODE-RED, 2024).

No contexto industrial, o Node-RED funciona como middleware entre dispositivos de campo e sistemas supervisórios, abstraindo a complexidade dos protocolos de comunicação e permitindo a integração rápida de novos dispositivos ao pipeline de dados (RODRIGUES, 2024).

2.4 APIs RESTful para Sistemas IoT

De acordo com Rodrigues (2024), a expansão de funcionalidades em sistemas IoT depende diretamente da integração com APIs HTTP, que possibilitam maior flexibilidade e controle remoto dos dispositivos. A utilização de arquiteturas REST garante interoperabilidade entre diferentes plataformas e tecnologias.

Aguiar (2023) complementa que a Internet das Coisas pode ser descrita como uma rede de objetos físicos conectados à internet, capazes de interagir por meio de sensores e softwares, sendo as APIs RESTful fundamentais para conectar o mundo físico ao digital.

2.5 Alarmística Industrial e Norma ISA-101

A norma ISA-101 (*Human Machine Interfaces for Process Automation Systems*) estabelece diretrizes para o projeto de interfaces industriais de alta performance. Entre seus princípios estão o uso criterioso de cores, a hierarquia de telas e o destaque adequado para alarmes e situações anormais (ISA, 2015).

A gestão de alarmes em sistemas supervisórios envolve a classificação por prioridade, o reconhecimento por parte do operador e o registro histórico para análise de tendências e diagnóstico de falhas.

3 Metodologia

O sistema foi desenvolvido seguindo uma arquitetura em quatro camadas, conforme ilustrado na Figura 1. O pipeline completo de dados segue o fluxo: OPC-UA Server → Node-RED → Backend Express → Frontend Next.js.

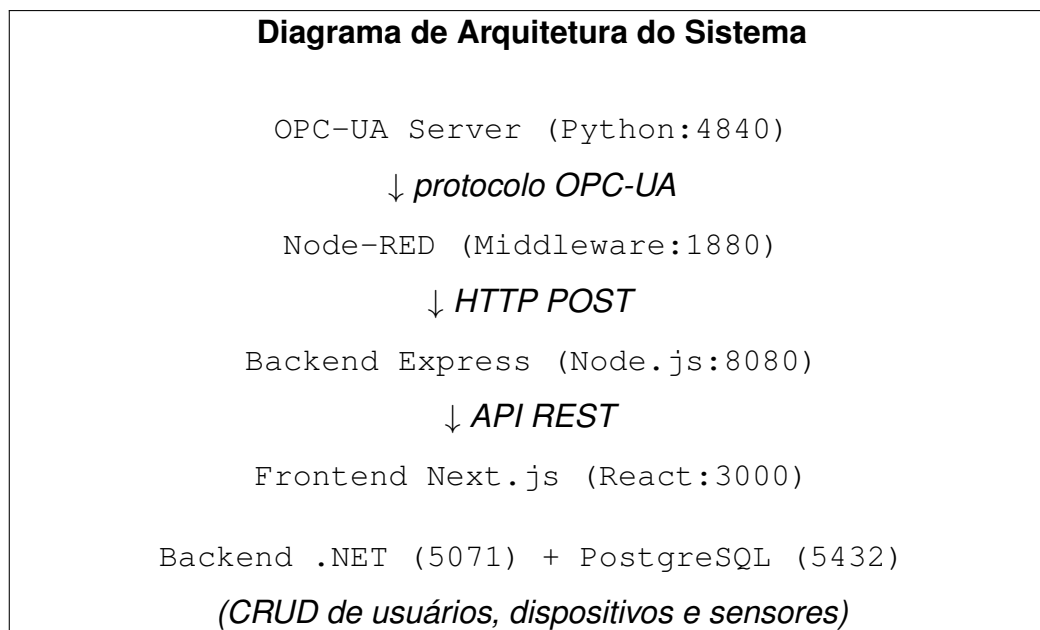


Figura 1: Arquitetura geral do sistema supervisor Web.

3.1 Camada 1 – Servidor OPC-UA (Python)

O servidor OPC-UA foi implementado em Python utilizando a biblioteca `opcua` (`python-opcua`).

O servidor simula um processo industrial com três variáveis:

- **Temperatura** (`ns=2; i=2`, Float): variação aleatória em torno de 25°C, com limites entre 5°C e 95°C;
- **Pressão** (`ns=2; i=3`, Float): variação aleatória em torno de 2,5 bar, com limites entre 0,2 e 6,0 bar;
- **Status de Operação** (`ns=2; i=4`, Boolean): estado ligado/desligado com probabilidade de alternância de 5%.

O servidor publica os valores atualizados a cada 1 segundo no endpoint `opc.tcp://localhost` permitindo leitura e escrita por clientes OPC-UA.

3.2 Camada 2 – Middleware Node-RED

O Node-RED atua como middleware de aquisição de dados, executando fluxos que:

1. Conectam-se ao servidor OPC-UA como cliente;

2. Leem as variáveis de processo (Temperatura, Pressão e Status) a cada 2 segundos;
3. Formatam os dados em objetos JSON com campos `tag`, `valor`, `unidade` e `nodeId`;
4. Envia os dados ao backend Express via requisições HTTP POST para o endpoint `/iot`.

Os fluxos foram configurados utilizando os nós `node-red-contrib-opcua` para comunicação OPC-UA e nós nativos `http request` para o envio dos dados.

3.3 Camada 3 – Backend Express.js

O backend foi implementado em Node.js com o framework Express.js, executando na porta 8080. Suas responsabilidades incluem:

- **Recepção de dados** (POST `/iot`): recebe leituras do Node-RED e armazena em memória;
- **Armazenamento de histórico**: mantém até 500 leituras por tag para consulta posterior;
- **Gerenciamento de alarmes**: verifica limites configurados e gera alarmes automáticos;
- **API de consulta**: endpoints GET para valores atuais (`/current`), histórico (`/history/:tag`), alarmes (`/alarms`) e status (`/status`);
- **Escrita OPC-UA**: endpoint POST `/write` para envio de comandos diretamente ao servidor OPC-UA via biblioteca `node-opcua-client`.

Os limites de alarme configurados são apresentados na Tabela 1.

Tabela 1: Limites de alarme configurados no backend.

Tag	Limite Inferior	Limite Superior	Unidade
Temperatura	10	80	°C
Pressão	0,5	5,0	bar

3.4 Camada 4 – Frontend Next.js

A interface Web foi desenvolvida utilizando o framework Next.js (React) com TypeScript, executando na porta 3000. O sistema implementa as seguintes telas:

- **Dashboard Principal** (/): exibe valores atuais de temperatura, pressão e status de operação em tempo real, com gráficos de tendência (últimos 60 pontos) e banner de alarmes ativos. Atualização automática a cada 3 segundos;
- **Histórico de Processo** (/historico): permite seleção de tag (Temperatura ou Pressão), período (1, 5, 10 minutos ou tudo), tipo de gráfico (área ou linha), exibindo estatísticas (mínimo, média e máximo) e tabela de dados brutos;
- **Alarmes e Diagnóstico** (/alarmes): apresenta resumo de alarmes (total, ativos, reconhecidos, resolvidos), filtros por prioridade, status e tag, com funcionalidades de reconhecimento individual e em lote;
- **Dispositivos** (/devices): CRUD de dispositivos e sensores integrado ao backend .NET;
- **Usuários** (/users): gerenciamento de usuários com autenticação JWT;
- **Login** (/login): autenticação via JWT com chave simétrica.

A biblioteca `recharts` foi utilizada para renderização dos gráficos de tendência (`AreaChart` e `LineChart`), e o gerenciamento de estado de autenticação utiliza a Context API do React.

3.5 Camada Complementar – Backend .NET e PostgreSQL

Um backend em ASP.NET Core (.NET 9) executa na porta 5071 e fornece uma API RESTful para operações CRUD de:

- Usuários (registro, login com JWT);
- Dispositivos (cadastro de equipamentos industriais);
- Sensores e Atuadores (associados a dispositivos).

O banco de dados PostgreSQL 15 é executado em container Docker, armazenando os dados cadastrais do sistema.

3.6 Tecnologias Utilizadas

A Tabela 2 apresenta o resumo das tecnologias empregadas no desenvolvimento do sistema.

Tabela 2: Tecnologias utilizadas no projeto.

Camada	Tecnologia	Porta
Servidor OPC-UA	Python + python-opcua	4840
Middleware	Node-RED v4.1.8	1880
Backend IoT	Node.js + Express.js	8080
Frontend	Next.js + React + TypeScript	3000
Backend CRUD	ASP.NET Core (.NET 9)	5071
Banco de Dados	PostgreSQL 15 (Docker)	5432
Gráficos	Recharts	—
Autenticação	JWT (HS256)	—

4 Resultados e Discussões

O sistema supervisor Web foi implementado com sucesso, atendendo aos requisitos propostos pela disciplina de Interfaces Industriais. Os resultados obtidos são apresentados a seguir.

4.1 Pipeline de Aquisição de Dados

O pipeline completo OPC-UA → Node-RED → Express → Next.js foi validado com dados simulados. O servidor OPC-UA gera valores de temperatura e pressão com variação aleatória, simulando um processo industrial. Os dados são adquiridos pelo Node-RED a cada 2 segundos e encaminhados ao backend Express via HTTP POST.

A Tabela 3 apresenta um exemplo de leitura dos sensores em um instante de operação.

Tabela 3: Exemplo de valores de processo monitorados em tempo real.

Variável	Valor	Unidade	Nodetd OPC-UA
Temperatura	32,47	°C	ns=2;i=2
Pressão	2,83	bar	ns=2;i=3
Status	Ligado	–	ns=2;i=4

4.2 Dashboard de Monitoramento em Tempo Real

O dashboard principal apresenta os valores atuais das variáveis de processo em cards informativos, acompanhados de gráficos de tendência que exibem o comportamento recente das variáveis. A atualização automática a cada 3 segundos garante que o operador tenha visibilidade contínua do estado do processo.

Os gráficos de tendência utilizam a biblioteca Recharts, renderizando componentes `AreaChart` para temperatura e pressão com preenchimento gradiente, facilitando a identificação visual de desvios e tendências.

4.3 Sistema de Alarmística

O sistema de alarmes implementa verificação automática de limites para temperatura (10°C a 80°C) e pressão (0,5 a 5,0 bar). Quando uma variável ultrapassa os limites configurados, um alarme é gerado automaticamente com as seguintes informações:

- Tag da variável, tipo (HIGH/LOW) e prioridade;
- Valor lido e limite violado;
- Timestamp da ocorrência;
- Estado de reconhecimento e resolução.

A tela de alarmes permite ao operador visualizar alarmes ativos, reconhecê-los individualmente ou em lote, e consultar o histórico de alarmes resolvidos. Filtros por prioridade, status e tag facilitam a análise em cenários com múltiplos alarmes simultâneos.

A resolução automática dos alarmes ocorre quando a variável retorna à faixa normal de operação, seguindo o princípio de *return-to-normal* comum em sistemas supervisórios industriais.

4.4 Histórico de Processo

A tela de históricos permite a consulta de dados armazenados com seleção de variável, período de visualização e tipo de gráfico. O sistema calcula e exibe estatísticas descritivas (valor mínimo, médio e máximo) para o período selecionado, além de disponibilizar uma tabela com os dados brutos das últimas 20 leituras.

O backend armazena até 500 leituras por tag em memória, permitindo a análise de tendências para intervalos de até aproximadamente 17 minutos (considerando a taxa de aquisição de 2 segundos do Node-RED).

4.5 Escrita OPC-UA via Backend

O backend implementa capacidade de escrita direta no servidor OPC-UA por meio do endpoint `POST /write`, utilizando a biblioteca `node-opcua-client`. Esta funcionalidade permite o envio de comandos de controle ao processo simulado, como alteração de setpoints e acionamento de atuadores, viabilizando a operação bidirecional do sistema supervisório.

4.6 Discussão

O sistema desenvolvido demonstra que é viável construir um supervisório industrial funcional utilizando exclusivamente tecnologias Web open-source. A arquitetura em camadas garante separação de responsabilidades e permite a substituição individual de componentes sem impacto no restante do sistema.

A principal limitação identificada é o armazenamento de histórico em memória (volátil), que se perde ao reiniciar o backend. Em uma implementação de produção, recomenda-se a integração com um banco de dados de séries temporais, como InfluxDB ou TimescaleDB.

A taxa de aquisição de 2 segundos, embora adequada para a simulação proposta, pode ser insuficiente para processos industriais que exigem resolução temporal na ordem de milissegundos. A adoção de WebSockets em substituição ao polling HTTP representaria uma melhoria significativa na latência de atualização da interface.

5 Conclusão

Este trabalho apresentou o desenvolvimento de um sistema supervisório Web para monitoramento de processos industriais, integrando o protocolo OPC-UA com tecnologias modernas de desenvolvimento Web. O sistema implementa com sucesso um pipeline completo de aquisição de dados, desde a simulação de sensores até a visualização em dashboards interativos.

Os principais resultados alcançados foram:

- Implementação de um pipeline funcional OPC-UA → Node-RED → Express → Next.js;
- Dashboard de monitoramento em tempo real com gráficos de tendência;
- Sistema de alarmística com geração automática, reconhecimento e histórico;
- Telas de histórico de processo com análise estatística;
- CRUD completo de usuários, dispositivos e sensores;
- Capacidade de escrita bidirecional via OPC-UA.

Como trabalhos futuros, propõe-se:

- Integração com banco de dados de séries temporais para persistência do histórico;
- Implementação de comunicação via WebSockets para redução de latência;
- Adição de análise preditiva baseada em algoritmos de aprendizado de máquina;
- Implantação em ambiente de nuvem para acesso remoto;
- Integração com controladores industriais reais (CLPs) via OPC-UA.

Agradecimentos

Agradeço à instituição de ensino SENAI e ao professor orientador Gabriel Claro pela orientação e suporte no desenvolvimento deste trabalho, bem como pelo conteúdo ministrado na disciplina de Interfaces Industriais que fundamentou este projeto.

Referências

AGUIAR, Claudio. **API REST e Internet das Coisas**: conectando o mundo físico ao digital. DIO, 2023.

GALLOWAY, Brendan; HANCKE, Gerhard P. Introduction to Industrial Control Networks. **IEEE Communications Surveys & Tutorials**, v. 15, n. 2, p. 860–880, 2013.

INTERNATIONAL SOCIETY OF AUTOMATION (ISA). **ISA-101.01-2015**: Human Machine Interfaces for Process Automation Systems. ISA, 2015.

NODE-RED. **Node-RED**: Low-code programming for event-driven applications. OpenJS Foundation, 2024. Disponível em: <https://nodered.org/>. Acesso em: 20 maio 2025.

OPC FOUNDATION. **OPC Unified Architecture Specification**. 2017. Disponível em: <https://opcfoundation.org/about/opc-technologies/opc-ua/>. Acesso em: 20 maio 2025.

RODRIGUES, Thaís dos Santos. **Expansão de funcionalidades em sistema IoT de controle de acesso através de API HTTP**. Trabalho de Conclusão de Curso – Universidade Federal de Pernambuco, 2024.